

USER GUIDE

Modbus Simulator

Simulate Modbus TCP, RTU & ASCII slave devices — register maps, data types and byte order, imports, the request log and troubleshooting.

powered by **TamidaS**

www.tamidas.com

Contents

1. What the simulator does

2. Getting started

3. How data flows

Modbus TCP

Modbus RTU

Modbus ASCII

Inside the simulator

4. Slaves

Adding a slave

Modbus TCP settings

Modbus RTU settings

Modbus ASCII settings

Simulation settings

The slave tab

Several slaves at once

5. Address spaces

6. Holding and input registers

Table columns

Data types and byte order

Value modes

Editing, adding and deleting

7. Coils and discrete inputs

Bit modes

Packed view

8. 0-based and 1-based addressing

9. Writes from the master

10. Importing a register map

11. Request log

12. Function codes and exceptions

13. Testing with a master

14. Command line and data folder

15. Troubleshooting

1. What the simulator does

The simulator makes your PC behave like one or more Modbus slave devices, so a Modbus master — a SCADA system, gateway, PLC, data logger or your own firmware — can be developed and tested without the real equipment.

TamidaS - Modbus Slave Simulator
www.tamidas.com

● Energy meter 23 regs ● Pump controller 1 reg

Energy meter slave id 1 · TCP 0.0.0.0:1502 (reachable at 192.168.0.81:1502) · 23 registers · update every 1000 ms · live

Addressing **0-based** 1-based **Stop** **Edit slave** **Remove slave**

TCP listening on 0.0.0.0:1502 · 0 connected · slaves 1
requests 8 · responses 8 · exceptions 1 · ignored 0 · connections 8

Holding Registers FC3 read · FC6 / FC16 write **12** **Input Registers** FC4 read · read-only **2** **Coils** FC1 read · FC5 / FC15 write **5** **Discrete Inputs** FC2 read · read-only **4**

Holding registers 12

Read with FC3. Masters can write with FC6 (single) and FC16 (multiple); a written register switches to **fixed** mode so the simulation stops overwriting it. Values are encoded with the chosen data type and byte order; the *Raw words* column is what goes on the wire.

+ Add register Upload CSV / XLS Save changes

Address (0-based)	Name	Type	Byte order	Mode	Min	Max	Step	Dec	Unit	Value	Raw words (hex)	Write
10	hr_10	uint16	ABCD	fixed	0	65535	1	0		7	0007	FC6 / FC16
19000	Voltage_L1_N	float32	ABCD	fixed	228	232	1	1	V	230.5	4366 8000	FC6 / FC16
19002	Energy_total	float64	ABCD	increment	10000	999999	0.5	1	Wh	12392	40C8 3400 0000 0000	FC6 / FC16
19006	Temperature	int16	ABCD	random	-20	60	1	0	C	-10	FFF6	FC6 / FC16
19007	Status_word	uint16	ABCD	fixed	0	65535	1	0		1	0001	FC6 / FC16
19008	Power_signed	int32	ABCD	random	-5000	5000	1	0	W	-3505	FFFF F24F	FC6 / FC16
19010	Run_hours	uint32	ABCD	increment	0	429496	1	0	h	1093	0000 0445	FC6 / FC16
19012	Counter_64	int64	ABCD	increment	0	1000000	1	0		93	0000 0000 0000 005D	FC6 / FC16
19016	Serial_number	uint64	ABCD	fixed	0	1000000	1	0		123456789	0000 0000 075B CD15	FC6 / FC16
19020	Real_power_L1	float32	ABCD	random	1000	3000	1	1	W	1273.6	449F 3333	FC6 / FC16
19022	Real_power_L2	float32	ABCD	random	1000	3000	1	1	W	2157.5	4506 D800	FC6 / FC16

A TCP slave with its register map. Values change live; the request log at the bottom shows what the master asked for.

- **Three protocols.** Modbus TCP on any port, and Modbus RTU or Modbus ASCII over a serial line such as a USB-to-RS-485 adapter.
- **Any number of slaves.** Each slave has its own name, slave id, protocol and register map. Slaves can share a TCP port or a serial bus.
- **All four address spaces.** Holding registers, input registers, coils and discrete inputs.
- **Function codes** 1, 2, 3, 4, 5, 6, 7, 8, 15, 16, 17, 22 and 23, with correct exception responses for everything else.
- **Realistic data.** Integers and floats in any byte order, values that change by themselves, counters that increment, coils that toggle or pulse.
- **Register maps from files.** Upload a CSV or Excel register list; a template is included.
- **Full visibility.** Every request and response is logged as hex with its result.

About this simulator

×

Modbus slave simulator

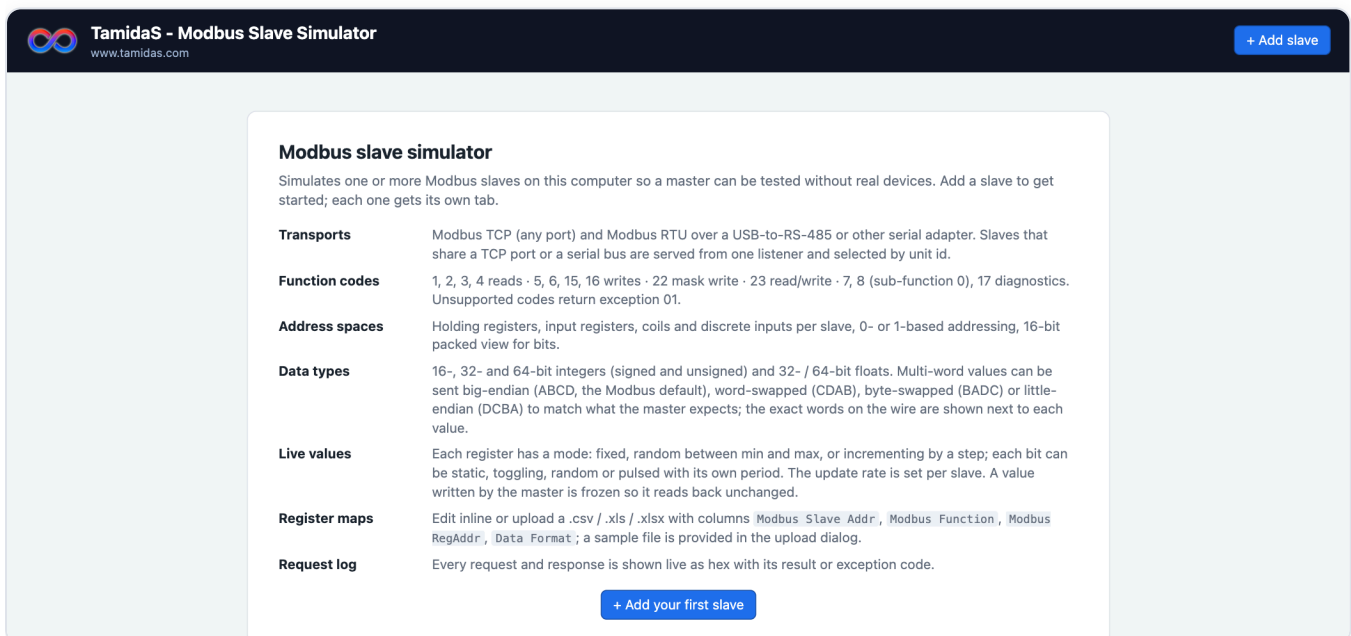
Simulates one or more Modbus slaves on this computer so a master can be tested without real devices. Add a slave to get started; each one gets its own tab.

Transports	Modbus TCP (any port) and Modbus RTU over a USB-to-RS-485 or other serial adapter. Slaves that share a TCP port or a serial bus are served from one listener and selected by unit id.
Function codes	1, 2, 3, 4 reads · 5, 6, 15, 16 writes · 22 mask write · 23 read/write · 7, 8 (sub-function 0), 17 diagnostics. Unsupported codes return exception 01.
Address spaces	Holding registers, input registers, coils and discrete inputs per slave, 0- or 1-based addressing, 16-bit packed view for bits.
Data types	16-, 32- and 64-bit integers (signed and unsigned) and 32- / 64-bit floats. Multi-word values can be sent big-endian (ABCD, the Modbus default), word-swapped (CDAB), byte-swapped (BADC) or little-endian (DCBA) to match what the master expects; the exact words on the wire are shown next to each value.
Live values	Each register has a mode: fixed, random between min and max, or incrementing by a step; each bit can be static, toggling, random or pulsed with its own period. The update rate is set per slave. A value written by the master is frozen so it reads back unchanged.
Register maps	Edit inline or upload a .csv / .xls / .xlsx with columns <code>Modbus Slave Addr</code> , <code>Modbus Function</code> , <code>Modbus RegAddr</code> , <code>Data Format</code> ; a sample file is provided in the upload dialog.
Request log	Every request and response is shown live as hex with its result or exception code.

The  button in the header shows this summary at any time.

2. Getting started

- 1 **Run the program.** On Windows double-click `tamidas-modbus-simulator.exe` ; on macOS or Linux unpack the `.tar.gz` and start the binary from a terminal (see [Command line](#) for the first-run notes). A console window opens and shows the web address, and your default browser opens the simulator. Keep the console window open; closing it stops the simulator.
- 2 **Add a slave.** On first start there are no slaves yet and the page shows an overview. Click `+ Add slave` in the header (or the button in the overview) and follow [Adding a slave](#).
- 3 **Load a register map.** Upload the device's register list (see [Importing a register map](#)) or add registers by hand.
- 4 **Connect your master** to the address shown in the slave tab and start polling. Watch the request log.



The start page before the first slave is added.

Where things are saved. Settings and register maps are stored in a `data` folder created next to the executable, so everything is back after a restart. Copy the executable to another folder to keep a separate setup.

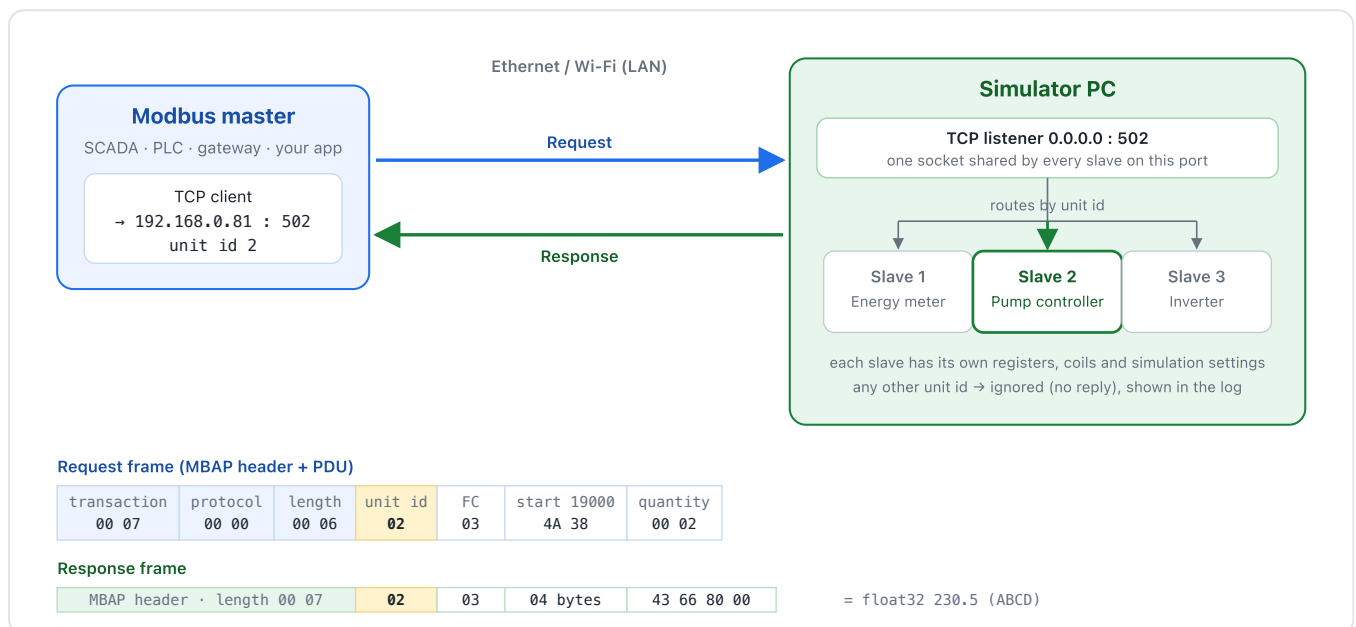
Web port. The interface uses port 8080. If that port is busy the next free port is used and printed in the console window. See [Command line](#) to change it.

3. How data flows between master and simulator

A Modbus master sends a request; the simulator answers on behalf of the slave whose id is in the request. The pictures below show that exchange for Modbus TCP and for Modbus RTU, and what happens inside the simulator in between.

Modbus TCP

The master opens a TCP connection to the simulator PC's address and port and sends requests over it. One listener serves every slave on that port; the **unit id** byte in each request selects the slave.

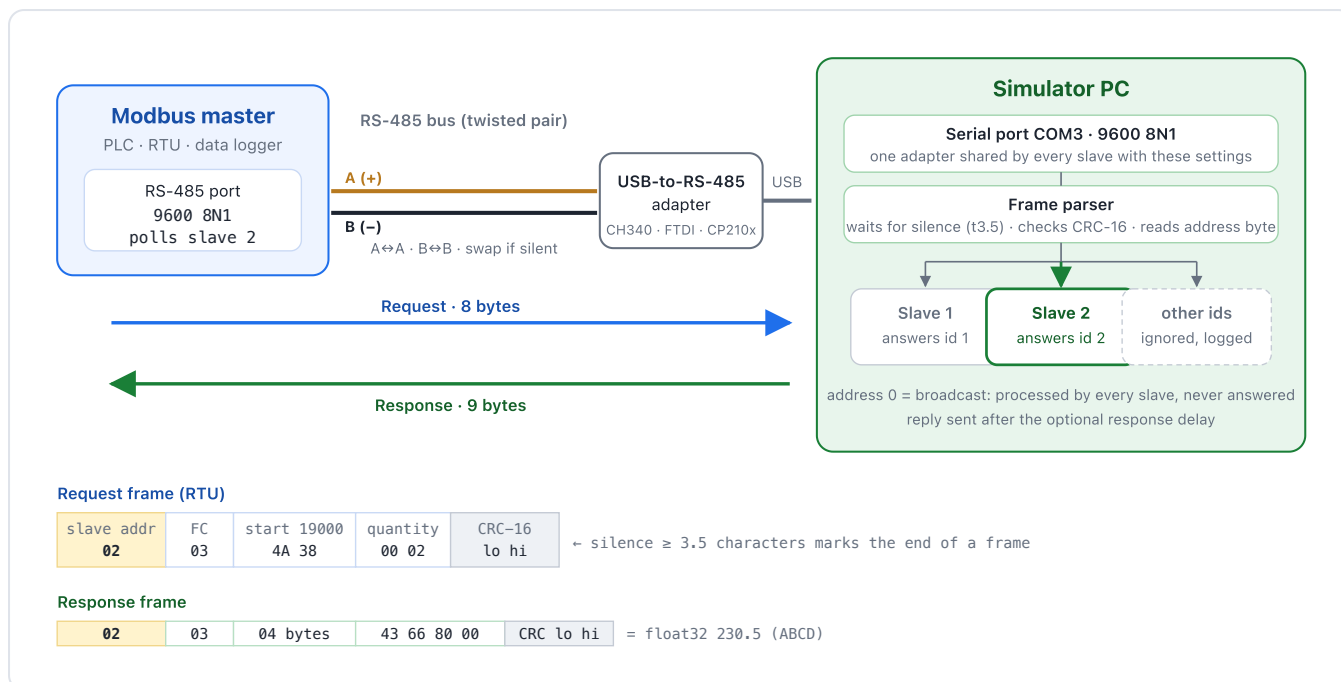


Modbus TCP: the master connects to the simulator's address and port. The listener hands each request to the slave whose id is in the unit-id byte, and the response travels back on the same connection.

- The master needs three things: the simulator PC's **IP address** (shown in the slave tab), the **port**, and the **unit id** equal to the slave id.
- Several masters can be connected at the same time; each connection is served independently.
- Slaves on different ports get their own listener; slaves on the same host and port share one.

Modbus RTU

With RTU there is no network: the master and the simulator PC are joined by a two-wire RS-485 bus through a USB adapter. Frames are raw bytes with a CRC, separated by silence, and the first byte is the slave address.



Modbus RTU: request and response share the same two wires, so only one device talks at a time. The simulator answers only the slave ids it hosts on that serial port.

- Line settings (baud rate, data bits, parity, stop bits) must be identical on both ends, otherwise frames arrive corrupted and show as *bad-crc* in the log.
- Because the bus is shared, the master must wait for each response before sending the next request; the simulator replies immediately unless a response delay is configured.
- Several simulated slaves on one adapter behave like several physical devices on the bus.

Modbus ASCII

Modbus ASCII is the second transmission mode of the Modbus serial line. The wiring, the bus rules and the slave addressing are those of [Modbus RTU](#); what changes is how a frame is written on the wire. Every byte is sent as two readable hexadecimal characters, a frame starts with a colon and ends with carriage return + line feed, and the checksum is an 8-bit LRC instead of the CRC-16. Older PLCs, some drives and devices behind modems or radio links use it because it tolerates long pauses between characters.

The same request — read 2 holding registers at 19000 from slave 2

Modbus RTU · 8 bytes, binary, ends with silence

slave addr	FC	start 19000	quantity	CRC-16	
02	03	4A 38	00 02	lo hi	← silence ≥ 3.5 characters marks the end

Modbus ASCII · 17 characters, text, framed by ':' and CR LF

start	slave addr	FC	start 19000	quantity	LRC	end	
:	02	03	4 A 3 8	0 0 0 2	7 7	CR LF	each byte is sent as two hex characters (0–9, A–F)

Response frame (ASCII) · 21 characters

start	slave addr	FC	byte count	data = float32 230.5	LRC	end
:	02	03	04	4 3 6 6 8 0 0 0	C E	CR LF

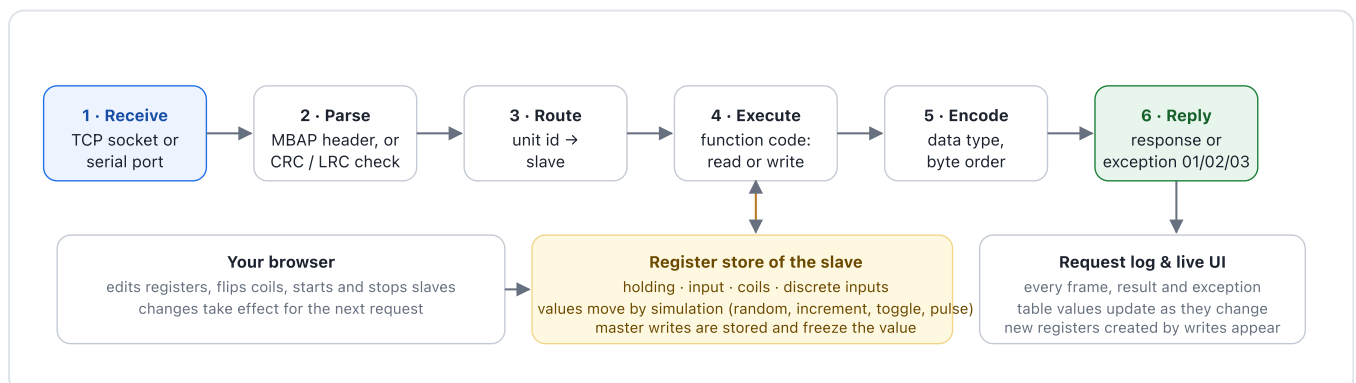
LRC = two's complement of the 8-bit sum of the bytes before it: 02+03+4A+38+00+02 = 89h → 100h – 89h = 77h. In the request log: :02034A38000277 → :02030443668000CE

Modbus ASCII carries exactly the same address, function code and data as RTU; only the framing and the checksum differ.

- A frame begins at `:` and ends at CR LF. A new colon always restarts reception, so a master that aborts a frame and starts again is understood.
- Characters may arrive up to **1 second** apart. A longer gap discards the partial frame, which is logged as *bad-lrc*.
- The standard line setting is **7 data bits, even parity, 1 stop bit** (7E1), because only ASCII characters travel on the line; many devices also allow 7O1, 7N2 or 8N1. It must match the master.
- Frames with a wrong LRC, an odd number of hex characters or non-hex characters are not answered and show as *bad-lrc* in the log. Lowercase hex is accepted; responses use uppercase.
- A frame is about twice as long as in RTU, so polling is slower at the same baud rate.
- RTU and ASCII cannot be mixed on one bus: every device on a serial port uses the same mode. Slave address 0 is a broadcast — processed, never answered — exactly as in RTU.

Inside the simulator

Whichever transport delivered the frame, the same steps produce the answer:



One pipeline for every protocol. Step 4 reads from or writes to the slave's register store, which the simulation and the web page also change.

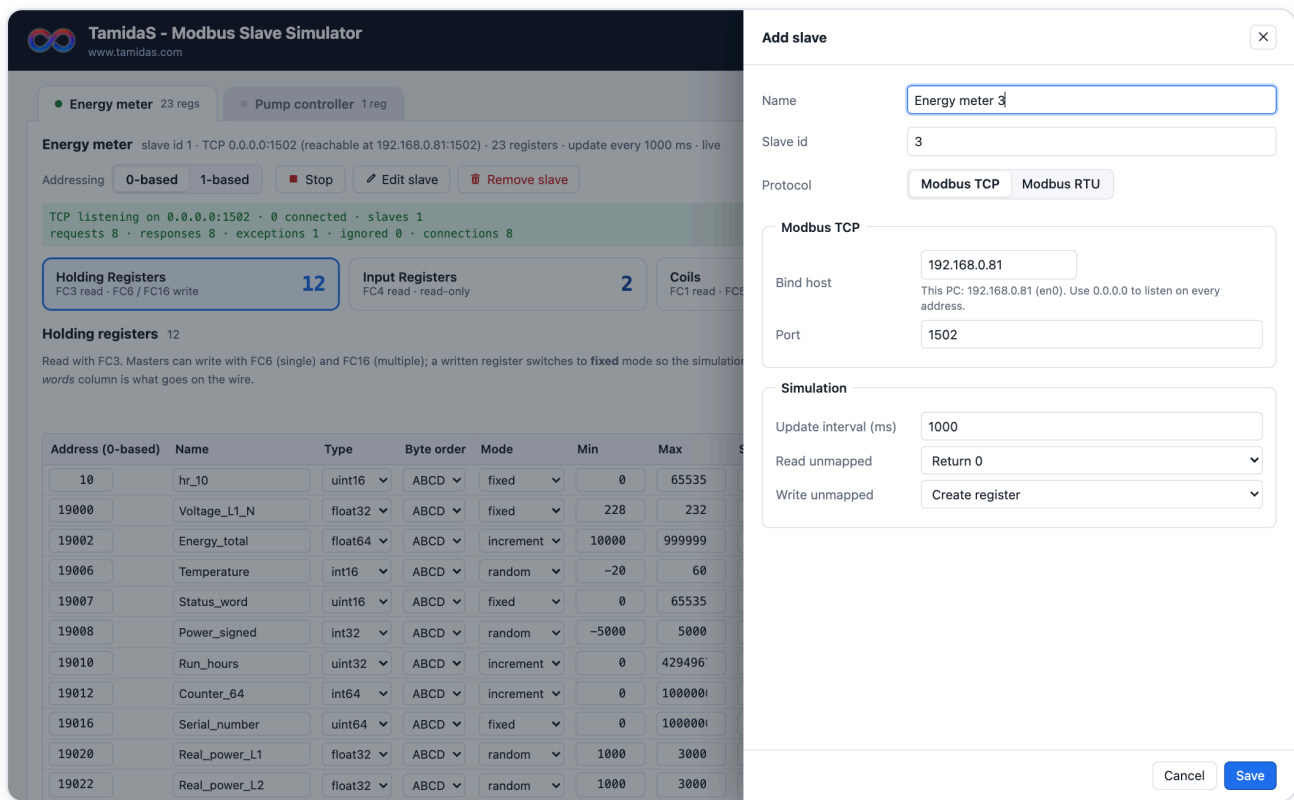
Step	What can go wrong	Seen as
2 · Parse	Wrong baud rate or parity, noise on the bus	<i>bad-crc</i> in the log, no reply
3 · Route	Master uses a slave id that is not hosted on that port or bus	<i>ignored</i> in the log, no reply — the master times out
4 · Execute	Unsupported function, undefined address, bad quantity	exception 01 / 02 / 03
5 · Encode	Byte order or type differs from what the master expects	reply is <i>ok</i> but the master shows a wrong value — compare with the <i>Raw words</i> column

4. Slaves

A slave is one simulated device. Every slave appears as a tab named after it.

Adding a slave

Click + Add slave. A panel slides in from the right.



Adding a Modbus TCP slave. The bind host is prefilled with this PC's network address.

Field	Meaning
Name	Shown on the tab. Anything you like, for example the device model.
Slave id	The Modbus unit identifier the master will address, 1–247. The next free id is suggested. It cannot be changed later; remove and re-add the slave instead.
Protocol	Modbus TCP , Modbus RTU or Modbus ASCII . The settings below change with the choice.

Press Save. The slave is created, its transport is started, and its tab opens. If the transport could not start — for example the port is in use — the slave is still created and the reason is shown in the tab.

Modbus TCP settings

Field	Meaning
Bind host	The address the slave listens on. The simulator detects the PC's network addresses and prefills the LAN address; this is also the address to enter in the master. Choose <code>0.0.0.0</code> to listen on every address of the PC, or <code>127.0.0.1</code> to accept connections from this PC only. Click the field to pick from the list.
Port	TCP port, normally <code>502</code> . No administrator rights are needed. Use another port if 502 is taken by other software.

Tip. Binding to a specific address stops working if the PC gets a new address from DHCP. `0.0.0.0` avoids that; the tab then shows the reachable address next to it.

Modbus RTU settings

The screenshot shows a 'Add slave' dialog box with the following fields and options:

- Name:** Energy meter 3
- Slave id:** 3
- Protocol:** Modbus TCP (selected), Modbus RTU
- Modbus RTU (serial / RS-485):**
 - Serial port:** (auto-detect USB adapter) with a refresh icon
 - Baud rate:** 9600
 - Data bits:** 8
 - Parity:** None
 - Stop bits:** 1
 - Response delay (ms):** 0
- Simulation:**
 - Update interval (ms):** 1000
 - Read unmapped:** Return 0
 - Write unmapped:** Create register

Buttons at the bottom: Cancel, Save.

RTU settings. The serial port list is read from the PC; press  after plugging in an adapter.

Field	Meaning
Serial port	The COM port of the USB-to-RS-485 adapter (Windows <code>COMx</code> , macOS <code>/dev/tty.usbseriall-...</code> , Linux <code>/dev/ttyUSB0</code>). <i>Auto-detect USB adapter</i> picks the first USB serial adapter found at start time.
Baud rate, Data bits, Parity, Stop bits	Must match the master exactly. Common meter settings are 9600 or 19200 baud, 8 data bits, no parity, 1 stop bit — written <code>9600 8N1</code> .
Response delay (ms)	Extra pause before each response. Leave at 0 unless you are testing a master's timeout handling or a slow device.

Wiring. Connect adapter A(+) to master A(+) and B(–) to B(–), plus GND if available. Labels differ between manufacturers, so if nothing arrives, swap A and B. Frames that arrive with a bad checksum show as *bad-crc* in the request log, which usually means wrong baud rate or parity.

Modbus ASCII settings

A Modbus ASCII slave uses the same serial settings as an RTU slave — serial port, baud rate, data bits, parity, stop bits and response delay — with these differences:

Field	Meaning
Data bits, Parity	Choosing Modbus ASCII switches the fields to the standard <code>7E1</code> (7 data bits, even parity, 1 stop bit) if they still hold the RTU default <code>8N1</code> , and back again when you return to RTU. Change them if your master uses something else, for example <code>8N1</code> or <code>7O1</code> .
Response delay (ms)	As for RTU. There is no silence requirement between ASCII frames, so 0 is fine for every master.

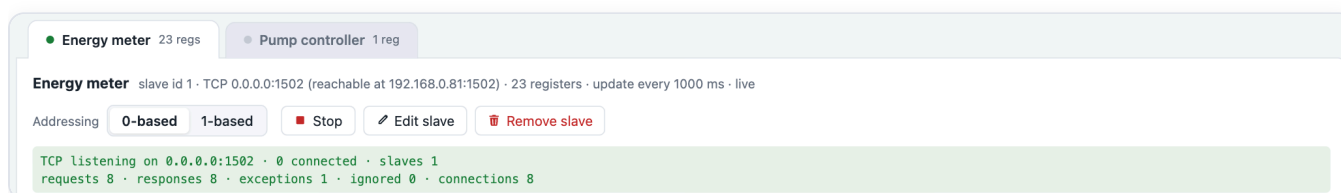
One mode per serial port. Slaves that share an adapter must all be RTU or all be ASCII, with identical line settings. A slave whose mode differs from the one already running on that port is not started and shows a conflict message. To test both modes, use two adapters or switch the slaves' protocol.

Simulation settings

These three settings are per slave and sit at the bottom of the Add / Edit panel.

Setting	Meaning
Update interval (ms)	How often registers in <i>random</i> and <i>increment</i> mode get a new value. Bits have their own period (see Bit modes).
Read unmapped	What a master gets when it reads an address you have not defined. Return 0 makes block reads over gaps succeed, like most meters. Exception 02 rejects the whole read with <i>illegal data address</i> , like a strict device.
Write unmapped	What happens when a master writes to an undefined address. Create register adds a new holding register or coil at that address with the written value. Exception 02 rejects the write.

The slave tab

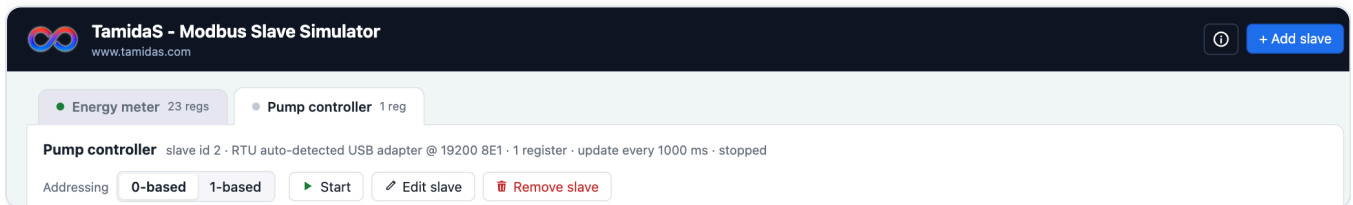


The tab header: name, summary line, addressing switch, Start/Stop, Edit and Remove, and the transport status.

- **Tab strip.** One tab per slave with a status dot: green when its transport is running, red when it failed, grey when stopped. The register count is shown next to the name.
- **Summary line.** Slave id, protocol and endpoint, number of registers, update interval and whether the transport is live.
- **Addressing.** Shows addresses 0-based or 1-based everywhere. See [Addressing](#).
- **Start / Stop** — one button that starts or stops this slave's transport. The label shows the action it will perform.
- **Edit slave** — opens the same panel as Add slave with the current values. Name, protocol, transport and simulation settings can all be changed; saving applies them and restarts the transport if it was running.
- **Remove slave** — deletes the slave and all its registers after confirmation.

Edit slave opens the same panel prefilled; only the slave id is locked.

- **Status line.** Green while listening (with request counters for TCP), red with the reason when the transport could not start, hidden when stopped.



An RTU slave that is stopped. Press Start after plugging in the adapter.

Several slaves at once

Add as many slaves as you need. How they share the hardware follows the Modbus rules:

- **Same TCP port.** Slaves with the same bind host and port share one listener. The unit id in each request selects the slave. A request whose unit id matches none of them — including 0 and 255 — is not answered and appears as *ignored* in the log, so set the master's unit id to the slave id.
- **Different TCP ports.** Each gets its own listener.
- **Same serial port.** Slaves with the same port, the same mode (RTU or ASCII) and identical line settings share the adapter, exactly like several devices on one RS-485 bus. Frames for other slave ids are ignored. If the line settings differ, the second slave is not started and shows a conflict message.
- **TCP and RTU together** are independent and can run side by side.

A slave speaks one protocol. To expose the same register map over both TCP and RTU, create two slaves and import the same file into each.

5. Address spaces

Each slave has the four Modbus address spaces. The cards at the top of the tab select which one the table shows and display the register count in each.

Holding Registers
FC3 read · FC6 / FC16 write
12

Input Registers
FC4 read · read-only
2

Coils
FC1 read · FC5 / FC15 write
5

Discrete Inputs
FC2 read · read-only
4

Click a card to show that space. The card also tells you which function codes read and write it.

Space	Read	Write	Content
Holding registers	FC3	FC6 (one), FC16 (many), FC22, FC23	16-bit words; multi-word values span consecutive addresses
Input registers	FC4	read-only	16-bit words
Coils	FC1	FC5 (one), FC15 (many)	single bits
Discrete inputs	FC2	read-only	single bits

Addresses are independent per space and per slave: holding register 0 and coil 0 are different things, and slave 1's register 100 has nothing to do with slave 2's.

6. Holding and input registers

Holding registers 12

Read with FC3. Masters can write with FC6 (single) and FC16 (multiple); a written register switches to **fixed** mode so the simulation stops overwriting it. Values are encoded with the chosen data type and byte order; the *Raw words* column is what goes on the wire.

[+ Add register](#)[Upload CSV / XLS](#)[Save changes](#)

Address (0-based)	Name	Type	Byte order	Mode	Min	Max	Step	Dec	Unit	Value	Raw words (hex)	Write
10	hr_10	uint16 ▾	ABCD ▾	fixed ▾	0	65535	1	0		7	0007	FC6 / FC16
19000	Voltage_L1_N	float32 ▾	ABCD ▾	fixed ▾	228	232	1	1	V	230.5	4366 8000	FC6 / FC16
19002	Energy_total	float64 ▾	ABCD ▾	increment ▾	10000	999999	0.5	1	Wh	12392	40C8 3400 0000 0000	FC6 / FC16
19006	Temperature	int16 ▾	ABCD ▾	random ▾	-20	60	1	0	C	-10	FFF6	FC6 / FC16
19007	Status_word	uint16 ▾	ABCD ▾	fixed ▾	0	65535	1	0		1	0001	FC6 / FC16
19008	Power_signed	int32 ▾	ABCD ▾	random ▾	-5000	5000	1	0	W	-3505	FFFF F24F	FC6 / FC16
19010	Run_hours	uint32 ▾	ABCD ▾	increment ▾	0	429496	1	0	h	1093	0000 0445	FC6 / FC16
19012	Counter_64	int64 ▾	ABCD ▾	increment ▾	0	1000000	1	0		93	0000 0000 0000 005D	FC6 / FC16
19016	Serial_number	uint64 ▾	ABCD ▾	fixed ▾	0	1000000	1	0		123456789	0000 0000 075B CD15	FC6 / FC16
19020	Real_power_L1	float32 ▾	ABCD ▾	random ▾	1000	3000	1	1	W	1273.6	449F 3333	FC6 / FC16
19022	Real_power_L2	float32 ▾	ABCD ▾	random ▾	1000	3000	1	1	W	2157.5	4506 D800	FC6 / FC16
19050	Frequency	float32 ▾	ABCD ▾	random ▾	49.95	50.05	1	2	Hz	50.01	4248 0A3D	FC6 / FC16

The register table. Every cell is editable; values and raw words update live.

Table columns

Column	Meaning
Address	Start address of the value. A 32-bit value occupies this address and the next one, a 64-bit value four addresses. Shown 0- or 1-based according to the addressing switch.
Name	Free text to identify the register.
Type	Data type, see below.
Byte order	How multi-word values are arranged, see below.
Mode	How the value changes over time, see Value modes .
Min / Max	Range for random mode and wrap-around limits for increment mode.
Step	Amount added on each update in increment mode.
Dec	Decimal places kept when generating values.
Unit	Engineering unit, for your reference only.
Value	The current value. Type a value to set it; the register switches to <i>fixed</i> so it stays.
Raw words (hex)	The 16-bit words exactly as they are sent to the master, after type conversion and byte ordering.
Write	Which function codes can write this space, or <i>read-only</i> .

Data types and byte order

Type	Words	Range
int16 / uint16	1	−32 768...32 767 / 0...65 535
int32 / uint32	2	$\pm 2.1 \times 10^9$ / 0... 4.29×10^9
int64 / uint64	4	64-bit integers
float32	2	IEEE 754 single precision
float64	4	IEEE 754 double precision

Modbus itself only knows 16-bit words, so devices differ in how they split larger values. The **byte order** column reproduces the device you are simulating. With the bytes of a 32-bit value called A B C D (A most significant):

Byte order	Also called	Words on the wire
ABCD	big-endian, "Modbus standard"	AB CD
CDAB	word-swapped, little-endian words	CD AB
BADC	byte-swapped	BA DC
DCBA	little-endian	DC BA

Finding the right byte order. If the master shows nonsense for a float, look at the *Raw words* column and try another byte order until the master shows the value you typed. For 230.5 as float32 the words are 4366 8000 in ABCD and 8000 4366 in CDAB.

Value modes

Mode	Behaviour
fixed	The value never changes by itself. Typing a value sets this mode automatically.
random	Every update interval a new value between Min and Max is generated, rounded to Dec decimals. Good for voltages, currents and temperatures.
increment	Every update interval Step is added. When the value passes Max it wraps to Min. Good for energy and run-hour counters.

The update interval is a per-slave setting (see [Simulation settings](#)).

Editing, adding and deleting

+ Add register
Upload CSV / XLS
Save changes

The toolbar above each table.

- **Editing.** Change any cell directly in the table. Edited rows are highlighted and the **Save changes** button shows how many rows are pending. Press it, or **Ctrl + S** (**⌘ + S** on a Mac), to apply them. Live values keep updating rows you are not editing.
- **+ Add register** adds a row after the last one, at the next free address, copying the type and byte order of the previous row. Edit it and save.
- **X** at the end of a row deletes it after confirmation. Deletion is immediate and does not need Save.
- **Upload CSV / XLS** imports a whole map, see [Importing a register map](#).

Overlaps. If two registers cover the same address, a warning is shown above the table and the later definition wins on the wire. Fix the addresses or types so they do not overlap.

Input registers 2

Read with FC4. Input registers are read-only; there is no Modbus write function for this space, so writes are rejected with exception 01.

+ Add register

Upload CSV / XLS

Save changes

Address (0-based)	Name	Type	Byte order	Mode	Min	Max	Step	Dec	Unit	Value	Raw words (hex)	Write
0	Frequency	float32 ▾	ABCD ▾	random ▾	49.95	50.05	1	2	Hz	50.03	4248 1EB8	read-only ✕
2	Firmware_version	uint16 ▾	ABCD ▾	fixed ▾	0	65535	1	0		102	0066	read-only ✕

Input registers use the same table; they are read-only for the master.

7. Coils and discrete inputs

Coils (FC1) and discrete inputs (FC2) are single bits. Their table is simpler: an address, a name, a switch showing the current value, a mode with its parameters, and the write access.

Bits 5

Bits. Coils (FC1) are read/write: FC5 writes one, FC15 writes many. A master write drops the coil to **static** so a driven mode does not overwrite it on the next tick. Modes: **static** only changes when you click it or a master writes it · **toggle** flips every Period ms · **random** re-rolls every Period ms with probability Prob of being 1 · **pulse** rests, then flips for Width ms every Period ms.

RowsPacked

+ Add bitUpload CSV / XLSSave changes

Address (0-based)	Name	Value	Mode	Write
0	Breaker_closed	☒ 1	static	FC5 / FC15 ✕
1	Relay_fan	☐ 0	togglePeriod5000	FC5 / FC15 ✕
2	Relay_heater	☒ 1	pulsePeriod2000Width300	FC5 / FC15 ✕
3	Alarm_ack	☒ 1	static	FC5 / FC15 ✕
4	Door_contact	☒ 1	randomPeriod2000Prob0.3	FC5 / FC15 ✕

Coils. Click a switch to flip a bit immediately — no Save needed.

- **Switch.** Green with **1** means set, grey with **0** means clear. Clicking it changes the value at once; in a driven mode the new value is the starting point for the next cycle.
- adds a bit at the next free address.
- Coils can be written by the master with FC5 and FC15; discrete inputs are read-only and reject those with exception 01.

Bits 4

Bits. Discrete inputs (FC2) are read-only and reject FC5 / FC15 with exception 01. Drive them from the UI: **static**, **toggle** (flip every Period ms), **random** (probability Prob every Period ms) or **pulse** (flip for Width ms every Period ms).

RowsPacked

+ Add bitUpload CSV / XLSSave changes

Address (0-based)	Name	Value	Mode	Write
0	Door_open	☐ 0	randomPeriod1000Prob0.3	read-only ✕
1	Overtemp_trip	☐ 0	static	read-only ✕
2	Motor_running	☐ 0	togglePeriod3000	read-only ✕
3	Phase_loss	☐ 0	static	read-only ✕

Discrete inputs: same table, read-only for the master, driven from here.

Bit modes

Choosing a mode shows only the fields that mode uses.

Mode	Parameters	Behaviour
static	—	Never changes by itself. Only a click or a master write changes it. Use it for inputs you want to control by hand.
toggle	Period	Flips every <i>Period</i> ms: a square wave with 50 % duty. Period 5000 gives a 10 s cycle. Simulates a running/stopped contact.
random	Period, Prob	Every <i>Period</i> ms the bit is re-rolled: it becomes 1 with probability <i>Prob</i> (0–1), otherwise 0. Prob 0.3 means set about 30 % of the time.
pulse	Period, Width	Rests at its current value, then flips to the opposite value for <i>Width</i> ms, every <i>Period</i> ms. Width 0 means one simulation tick. Simulates an alarm blip or a button press.

Periods and widths are handled on a 100 ms clock, so 50 ms is the smallest useful step.

Packed view

The Rows / Packed switch, available for coils and discrete inputs, shows the bits the way FC1 and FC2 return them: one 16-bit word per row with bit 0 on the right, and the word value in hex.

Bits 5

Bits. Coils (FC1) are read/write: FC5 writes one, FC15 writes many. A master write drops the coil to **static** so a driven mode does not overwrite it on the next tick. Modes: **static** only changes when you click it or a master writes it · **toggle** flips every Period ms · **random** re-rolls every Period ms with probability Prob of being 1 · **pulse** rests, then flips for Width ms every Period ms.

Rows Packed
+ Add bit
Upload CSV / XLS
Save changes

Each row is one 16-bit word as returned by FC1. Bit 0 is on the right, matching the response byte order. Click any bit to flip it live; click an unmapped bit to add it.

Range	Bits 15 → 0	Word (hex)
0 – 15	11001	0x0019
16 – 31		0x0000

set (1)

clear (0)

unmapped

Packed view. Green cells are set, white are clear, grey dashed cells are addresses without a bit.

- Click a green or white cell to flip that bit.
- Click a grey cell to add a bit at that address.
- The **Word (hex)** column is what a master reading those 16 bits receives, which makes it easy to compare with a master's raw view.

8. 0-based and 1-based addressing

Modbus frames carry 0-based addresses, but many manuals and masters number registers from 1, or use the 40001 / 30001 convention. The switch in the tab header changes how addresses are displayed everywhere for that slave: the tables, the packed view and the request log.

The screenshot shows the 'Energy meter' interface for slave id 1. At the top, it indicates 'TCP 0.0.0.0:1502 (reachable at 192.168.0.81:1502) · 23 registers · update every 1000 ms · live'. Below this, there are tabs for '0-based' and '1-based' addressing, with '0-based' currently selected. To the right of the tabs are buttons for 'Stop', 'Edit slave', and 'Remove slave'. A green status bar shows 'TCP listening on 0.0.0.0:1502 · 0 connected · slaves 1 · requests 8 · responses 8 · exceptions 1 · ignored 0 · connections 8'. Below the status bar, there are four summary boxes: 'Holding Registers' (FC3 read · FC6 / FC16 write) with a count of 12, 'Input Registers' (FC4 read · read-only) with a count of 2, 'Coils' (FC1 read · FC5 / FC15 write) with a count of 5, and 'Discrete Inputs' (FC2 read · read-only) with a count of 4. Below these boxes, a section titled 'Holding registers 12' provides a detailed description: 'Read with FC3. Masters can write with FC6 (single) and FC16 (multiple); a written register switches to **fixed** mode so the simulation stops overwriting it. Values are encoded with the chosen data type and byte order; the *Raw* words column is what goes on the wire.' At the bottom right, there are buttons for '+ Add register', 'Upload CSV / XLS', and 'Save changes'.

With 1-based addressing the column header changes and every address is shown one higher.

- The setting only affects what you see. Nothing changes on the wire.
- Values typed into the Address column are interpreted in the selected base.
- Import and export files always use 0-based protocol addresses.
- For the 40001 convention enter `40001` in the master as register 1 (1-based) or 0 (0-based) here — the leading 4 only identifies the holding register space.

9. Writes from the master

Masters can write holding registers (FC6, FC16, FC22, FC23) and coils (FC5, FC15). The simulator applies the write and keeps what was written.

- A written register switches to **fixed** mode and a written coil to **static**, so the simulation does not overwrite the master's value on the next update. The row in the table updates immediately.
- Writing one word of a multi-word value — for example only the high word of a float32 — re-decodes the value from the combined words, exactly as a real device would.
- FC22 (mask write) applies the AND and OR masks to the current word.
- FC23 performs the write first and then the read.
- Writes to addresses that are not defined either create a new register or coil (named `hr_<address>` or `coil_<address>`) or return exception 02, depending on the slave's *Write unmapped* setting. Newly created rows appear in the table with a notice.
- Input registers and discrete inputs have no write function code; the request is answered with exception 01.

10. Importing a register map

Instead of typing registers, upload the device's register list. The importer reads `.csv`, `.xls` and `.xlsx` files and fills all four spaces of the slave whose tab is open in one go.

Import register map



Drag & drop a `.csv`, `.xls` or `.xlsx` file here
or [browse for a file](#)

Need a template? [Download the sample .csv](#) — one row per data format, all four spaces.

On import

Replace current map

Append to current map

Mandatory columns: `Modbus Slave Addr`, `Modbus Function` (1 coils · 2 discrete inputs · 3 holding · 4 input), `Modbus RegAddr`, `Data Format` (float, double, int, uint with `Data Len` 2/4/8, or bool).
Optional: `JSON KeyWord / Name`, `Data Len`, 4 Bytes Order (0=ABCD 1=DCBA 2=BADC 3=CDAB), `Unit`, `Min`, `Max`, `Mode`, `Step`, `Decimals`, `Value`, `Period`, `Prob`, `Width`. Addresses in the file are always 0-based protocol addresses. Slave ids in the file that are not simulated yet are added as new tabs. Rows whose `Data Source` is not Modbus are skipped.

Drag a file onto the dotted box or click to browse. A sample file is one click away.

Steps

- 1 Open the tab of the slave the file belongs to and click `Upload CSV / XLS` above any table. The dialog title names that slave.
- 2 Choose **Replace current map** to discard that slave's registers and load the file, or **Append to current map** to add the file's rows to them. Other slaves are never touched.
- 3 Drop the file on the dotted box, or click it to browse. The import runs at once and the dialog closes; a message summarises how many rows were imported and lists any that were skipped with the reason.

File format

The first row holds column names. Names are matched loosely — case, spaces and punctuation are ignored — and the header may be preceded by a few title rows.

Column	Required	Meaning
Modbus Function	yes	The read function code: 1 coil, 2 discrete input, 3 holding register, 4 input register.
Modbus RegAddr	yes	0-based start address.
Data Format	yes	float, double, int, uint, bool, or a simulator type name such as float32 or uint16.
Data Len	no	Size in bytes: 2, 4 or 8. Decides between int16 / int32 / int64 and float32 / float64. Defaults: float → float32, int / uint → 16-bit.
4 Bytes Order	no	Byte order: 0 or ABCD, 1 or DCBA, 2 or BADC, 3 or CDAB.
JSON KeyWord or Name	no	Register name.
Unit, Min, Max, Mode, Step, Decimals, Value	no	Simulation settings for registers, as in the table.
Period, Prob, Width	no	Bit mode parameters in ms / 0–1 / ms.
Data Source	no	If present, rows whose source does not contain "Modbus" are skipped. Useful for device export files that mix Modbus rows with others.
Modbus Slave Addr	ignored	Many device files carry a slave-address column. It is ignored: every row goes into the slave you are uploading to.

When Min, Max, Mode and Unit are missing, the importer guesses sensible values from the name: names containing *volt* get 228–232 V, *curr* 4–16 A, *freq* 49.95–50.05 Hz, *power* 1000–3000 W, *energy* an incrementing counter, *THD* 0.5–4 %.

Template. The *Download the sample .csv* link in the dialog gives a file with one row per data format across all four spaces. Open it in Excel, replace the rows with your device's list and upload it into the slave's tab.

Skipped rows. A row is skipped when a mandatory cell is empty, the function code is not 1–4, the address is not a number, or its Data Source is not Modbus. The message after import lists each skipped row with its reason. If no row can be imported, the dialog stays open and shows the problem, for example a missing mandatory column.

11. Request log

The request log at the bottom of the page shows every frame the simulator receives, on every transport and for every slave, newest first.

Request log 8

☐ Pause

Time	Via	Slave	FC	Address (0-based)	Qty / value	Result	Request	Response	
12:49:43.212	TCP	127.0.0.1:60548	1	2	0	4	ok	00 07 00 00 00 06 01 02 00 00 00 04	00 07 00 00 00 04 01 02
12:49:43.211	TCP	127.0.0.1:60547	9	6	10	1 · 0x0007 (7)	ok	00 07 00 00 00 06 09 06 00 0A 00 07	00 07 00 00 00 06 09 06
12:49:43.211	TCP	127.0.0.1:60546	1	3	0	200	exception 3 illegal data value	00 07 00 00 00 06 01 03 00 00 00 C8	00 07 00 00 00 03 01 83
12:49:43.210	TCP	127.0.0.1:60545	1	4	0	2	ok	00 07 00 00 00 06 01 04 00 00 00 02	00 07 00 00 00 07 01 04
12:49:43.210	TCP	127.0.0.1:60544	1	16	19000	2	ok	00 07 00 00 00 0B 01 10 4A 38 00 02 04 43 66 80 00	00 07 00 00 00 06 01 10
12:49:43.209	TCP	127.0.0.1:60543	1	5	3	1 · ON	ok	00 07 00 00 00 06 01 05 00 03 FF 00	00 07 00 00 00 06 01 05
12:49:43.209	TCP	127.0.0.1:60542	1	1	0	8	ok	00 07 00 00 00 06 01 01 00 00 00 08	00 07 00 00 00 04 01 01
12:49:43.208	TCP	127.0.0.1:60541	1	3	19000	8	ok	00 07 00 00 00 06 01 03 4A 38 00 08	00 07 00 00 00 13 01 03

Request and response as hex, with the result. Exceptions and ignored frames are highlighted.

Column	Meaning
Time	Arrival time with milliseconds.
Via	TCP with the client address and port, RTU or ASCII.
Slave	Unit id in the request.
FC	Function code; hover for its name.
Address, Qty / value	Start address (in the selected base) and quantity, or the written value for single writes, masks for FC22.
Result	ok · exception N with its name · ignored when the unit id is not served on that transport · broadcast for unit id 0 (processed, never answered) · bad-crc for corrupted RTU frames · bad-lrc for ASCII frames with a wrong checksum, invalid characters or an inter-character timeout.
Request, Response	The complete frames in hex, including the TCP header or the RTU CRC. Modbus ASCII frames are shown as the text that travelled on the line, for example <code>:02034A38000277</code> (the trailing CR LF is not shown).

Pause

 stops the list from scrolling while you read;

Clear

 empties it. The last 300 requests are kept.

12. Function codes and exceptions

FC	Name	Notes
1	Read Coils	1–2000 bits per request
2	Read Discrete Inputs	1–2000 bits
3	Read Holding Registers	1–125 registers
4	Read Input Registers	1–125 registers
5	Write Single Coil	value <code>FF00</code> = on, <code>0000</code> = off
6	Write Single Register	
7	Read Exception Status	returns 0 (serial line)
8	Diagnostics	sub-function 0 (return query data) only
15	Write Multiple Coils	1–1968 bits
16	Write Multiple Registers	1–123 registers
17	Report Server ID	returns the slave id and a run indicator
22	Mask Write Register	(current AND and_mask) OR (or_mask AND NOT and_mask)
23	Read/Write Multiple Registers	write first, then read

Exception	Meaning	When the simulator sends it
01	Illegal function	Unsupported function code, write to a read-only space, diagnostics sub-function other than 0
02	Illegal data address	Read or write touching an undefined address while the slave is set to reject it; address beyond 65535
03	Illegal data value	Quantity out of range, byte count mismatch, coil value other than FF00/0000

13. Testing with a master

Modbus TCP

- 1 Add a TCP slave. Note the address in the summary line, for example `192.168.0.81:502`.
- 2 In the master, create a TCP connection to that IP and port, with unit id equal to the slave id.
- 3 Poll, for example FC3 from address 19000 for 26 registers (13 floats). The request log shows each poll and the values are those in the table.

If the master is on another PC, allow the port through the Windows firewall when asked, or add a rule for the executable. The unit id in the master must equal the slave id; other ids are ignored.

Modbus RTU

- 1 Plug in the USB-to-RS-485 adapter and install its driver if Windows does not recognise it. Note its COM port in Device Manager.
- 2 Add an RTU slave with that port and the master's line settings, then Start it. The tab shows *RTU listening on COMx*.
- 3 Wire A to A and B to B, and poll from the master with the slave id.
- 4 No reply: swap A and B. *bad-crc* in the log: baud rate or parity mismatch. *ignored*: the master uses a different slave id.

Modbus ASCII

- 1 Connect the adapter as for RTU. In the master, select the **ASCII** transmission mode and note its line settings — usually `9600 7E1`.
- 2 Add a slave with protocol **Modbus ASCII**, the same port and line settings, then Start it. The tab shows *ASCII listening on COMx*.
- 3 Poll from the master. The request log shows the frames as text, which makes them easy to compare with the master's own trace or with a serial terminal.
- 4 *bad-lrc* in the log: line settings differ (7 versus 8 data bits is the usual cause) or the master is actually sending RTU. Nothing in the log at all: wiring, or the master is set to RTU and its binary frames contain no colon.

14. Command line and data folder

```
tamidas-modbus-simulator.exe          start and open the browser
tamidas-modbus-simulator.exe --no-open start without opening the browser
tamidas-modbus-simulator.exe --help   show the options

set PORT=9000 & tamidas-modbus-simulator.exe          web interface on another port
set MODBUS_SIM_DATA=D:\sim\data & tamidas-modbus-simulator.exe use another data folder

macOS / Linux – the same options, from a terminal:
./tamidas-modbus-simulator-macos-arm64 --no-open
PORT=9000 ./tamidas-modbus-simulator-linux-x64
MODBUS_SIM_DATA=/opt/sim/data ./tamidas-modbus-simulator-linux-x64
```

Build	For
<code>tamidas-modbus-simulator.exe</code>	Windows 10/11, 64-bit
<code>tamidas-modbus-simulator-macos-arm64</code>	Macs with Apple silicon (M1 and later) — portable binary
<code>tamidas-modbus-simulator-macos-x64</code>	Intel Macs — portable binary
<code>tamidas-modbus-simulator-<version>-macos-arm64.pkg</code>	Apple silicon — installer package
<code>tamidas-modbus-simulator-<version>-macos-x64.pkg</code>	Intel Macs — installer package
<code>tamidas-modbus-simulator-linux-x64</code>	64-bit Linux on Intel/AMD (Ubuntu, Debian, Fedora, ...)
<code>tamidas-modbus-simulator-linux-arm64</code>	64-bit Linux on ARM (Raspberry Pi 4/5 with a 64-bit OS, ARM servers)

macOS installer package (.pkg). Double-click the `.pkg`, click through the installer (it asks for your Mac password, as every installer does), then open Terminal and type `tamidas-modbus-simulator`. The package is signed and notarised by Apple and carries its notarisation ticket, so it installs without any security prompt, even offline. The command is placed in `/usr/local/bin`; settings and register maps go to `~/Library/Application Support/tamidas-modbus-simulator`. To remove it: `sudo rm /usr/local/bin/tamidas-modbus-simulator`.

First run on macOS (portable binary). The easiest way is the one-line installer, which needs no confirmation: open Terminal and run `curl -fsSL https://raw.githubusercontent.com/TamidaSRepo/modbus-simulator/main/install.sh | sh`, then start `tamidas-modbus-simulator`. If you unpacked the archive yourself, start the binary from Terminal (`./tamidas-modbus-simulator-macos-arm64`) or double-click it in Finder. The macOS builds are signed with the TamidaS Developer ID and notarised by Apple, so Gatekeeper lets them run; if macOS asks once whether to open a file downloaded from the internet, click *Open*. The first start needs an internet connection for that check. Use the `arm64` build on Apple silicon and the `x64` build on an Intel Mac.

First run on Linux. The same installer works on Linux: `curl -fsSL https://raw.githubusercontent.com/TamidaSRepo/modbus-simulator/main/install.sh | sh`. If you unpacked the archive yourself and the file is not executable, run `chmod +x` on it. To use a serial adapter without root, add yourself to the `dialout` group and log in again. TCP ports below 1024 (such as 502) need `sudo setcap 'cap_net_bind_service=+ep' <binary>` or a port such as 1502.

File	Content
<code>data/config.json</code>	Slaves with their transport and simulation settings.
<code>data/registers.json</code>	All registers and their current values. Saved on every change and every 30 s.

Both files are plain JSON. Copy the `data` folder to move a setup to another PC, or keep several folders and point `MODBUS_SIM_DATA` at the one you need.

15. Troubleshooting

Windows shows a SmartScreen warning when starting the exe

The file is not code-signed. Choose *More info* → *Run anyway*. The program only runs locally and does not connect to the internet.

macOS says the app cannot be opened or the developer cannot be verified

The published builds are signed and notarised, so this should not happen with a file downloaded from this site. It appears when the download was damaged, when the Mac is offline during the first start (Gatekeeper fetches the notarisation ticket once), or when the file is not the original. Download the archive again and check its SHA-256 against `SHA256SUMS`, or use the installer script, which verifies the checksum for you. On an Intel Mac use the `x64` build; on Apple silicon the `arm64` build.

The settings are not next to the executable

When the folder holding the executable cannot be written (Homebrew, `/usr/local/bin`, Program Files, a .pkg install) the data folder moves to the user's application-data folder: `~/Library/Application Support/tamidas-modbus-simulator` on macOS, `~/.local/share/tamidas-modbus-simulator` on Linux, `%APPDATA%\TamidaS\modbus-simulator` on Windows. The console prints the folder in use at start; `MODBUS_SIM_DATA` overrides it.

On Linux the serial port cannot be opened ("permission denied")

Your user is not allowed to use the adapter. Add it to the `dialout` group (`sudo usermod -aG dialout $USER`), log out and in again, and Start the slave. The same applies to `/dev/ttyACM0` devices.

The tab says "Port 502 is already in use"

Another program (or another slave with a different bind host) is listening on that port. Edit the slave and pick another port, or stop the other program.

The master times out on TCP

Check that the slave is live (green dot), that the master uses the address shown in the summary line, and that the Windows firewall allows the port. Requests that reach the simulator always appear in the log, so an empty log means the connection never arrived.

Values read by the master are wrong

Compare the master's raw registers with the *Raw words* column. If the words match but the value does not, change the byte order or data type. If the words differ, the master is reading a different address — remember 0- vs 1-based.

The master reads a block and gets exception 02

The block covers an address you have not defined and the slave's *Read unmapped* is set to Exception 02. Add the missing registers or switch the setting to Return 0.

A value I set keeps changing

The register is in random or increment mode. Set the mode to fixed, or type the value — typing sets fixed automatically.

A Modbus ASCII master gets no answer, or the log shows bad-lrc

Both ends must use the same mode and the same line settings. *bad-lrc* with readable text in the Request column usually means a data-bits or parity mismatch (the standard for ASCII is 7E1); unreadable text means the master is sending RTU frames to an ASCII slave. An RTU slave that receives ASCII frames logs *bad-crc* instead. Switch the slave's protocol in  Edit slave — the serial port and register map are kept.

Auto-detect does not find my USB adapter

Auto-detect looks for common USB serial names at start time. Press ↻ in the port list, select the COM port explicitly and Start the slave again.

Where are the log and the settings after I closed the program?

Settings and registers are in the `data` folder next to the executable; the request log is not saved.